

Створення CI

Усе вийшло — у нас є Jenkins, що працює всередині EKS.

Давайте ускладнимо наш проєкт: підключимо репозиторій із Django-проєктом, запусим код і створимо базовий CI-процес.

Але, як і раніше, ми робимо все максимально автоматизовано — без ручної роботи, через **інфраструктуру як код**.

Тому допрацюємо наш Terraform-модуль:

Додамо **Jenkins Configuration-as-Code** (JCasC), щоб автоматично створювати доступи до git та pipeline.

Змусимо Jenkins **самостійно підключатися до Git**-репозиторію.

Описуємо **CI-процес у Jenkinsfile**, який буде виконуватись під час кожного пушу.

І все це — без ручного кліку в UI.

Таким чином, Jenkins не просто встановлений, а одразу готовий до роботи з нашим кодом.

Для цього доповнимо файл [jenkins.tf](#) наступним кодом:

```
resource "kubernetes_storage_class_v1" "ebs_sc" {
  metadata {
    name = "ebs-sc"
    annotations = {
      "storageclass.kubernetes.io/is-default-class" = "true"
    }
  }
}

storage_provisioner = "ebs.csi.aws.com"

reclaim_policy      = "Delete"
volume_binding_mode = "WaitForFirstConsumer"

parameters = {
  type = "gp3"
}
```

```
}
```

```
resource "kubernetes_service_account" "jenkins_sa" {  
  metadata {  
    name      = "jenkins-sa"  
    namespace = "jenkins"  
    annotations = {  
      "eks.amazonaws.com/role-arn" =  
aws_iam_role.jenkins_kaniko_role.arn  
    }  
  }  
  depends_on = [  
    helm_release.jenkins  
  ]  
}
```

```
resource "aws_iam_role" "jenkins_kaniko_role" {  
  name = "${var.cluster_name}-jenkins-kaniko-role"  
  
  assume_role_policy = jsonencode({  
    Version = "2012-10-17",  
    Statement = [  
      {  
        Effect = "Allow",  
        Principal = {  
          Federated = var.oidc_provider_arn  
        },  
        Action = "sts:AssumeRoleWithWebIdentity",  
        Condition = {  
          StringEquals = {  
            "${replace(var.oidc_provider_url, "https://", "")}:sub"  
= "system:serviceaccount:jenkins:jenkins-sa"  
          }  
        }  
      }  
    ]  
  })  
}
```

```
resource "aws_iam_role_policy" "jenkins_ecr_policy" {  
  name = "${var.cluster_name}-jenkins-kaniko-ecr-policy"  
  role = aws_iam_role.jenkins_kaniko_role.id
```

```

policy = jsonencode({
  Version = "2012-10-17",
  Statement = [
    {
      Effect = "Allow",
      Action = [
        "ecr:GetAuthorizationToken",
        "ecr:BatchCheckLayerAvailability",
        "ecr:PutImage",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:DescribeRepositories"
      ],
      Resource = "*"
    }
  ]
})
}

resource "helm_release" "jenkins" {
  name           = "jenkins"
  namespace      = "jenkins"
  repository     = "<https://charts.jenkins.io>"
  chart          = "jenkins"
  version        = "5.8.27"
  create_namespace = true

  values = [
    file("${path.module}/values.yaml")
  ]
}

```

Пояснення:

`aws_iam_role.jenkins_kaniko_role` — IAM-роль, яка буде використана Jenkins agent'ом (що запускає Kaniko, про це детальніше — *потім*) для доступу до Amazon ECR.

Вона надається через IRSA (IAM Roles for Service Accounts) конкретному Kubernetes-сервіс-акаунту `jenkins-sa`.

`assume_role_policy` — trust policy, яка вказує, що роль можна «припустити» (assume) лише через OIDC-ідентифікацію.

Це дозволяє Jenkins'у безпечно отримати тимчасові AWS credentials **без секретів чи доступних ключів**.

"Principal.Federated" — OIDC-провайдер, який створюється в модулі EKS (`aws_iam_openid_connect_provider`) і передається як змінна `var.oidc_provider_arn`.

"Condition.StringEquals" — обмежує використання ролі тільки сервіс-акаунтом `system:serviceaccount:jenkins:jenkins-sa`.

`aws_iam_role_policy.jenkins_ecr_policy` — прикріплена IAM-політика, яка дозволяє Jenkins'у:

автентифікуватися до ECR (`ecr:GetAuthorizationToken`);

переглядати репозиторії;

пушити Docker-образи (`upload`, `put image`).

`kubernetes_service_account.jenkins_sa` — створює сервіс-акаунт `jenkins-sa` в namespace `jenkins`, який пов'язаний з IAM-роллю `jenkins_kaniko_role` через анотацію:

```
eks.amazonaws.com/role-arn = <ARN полі>.
```

Вищеописаний код — це і є суть **IRSA (IAM Roles for Service Accounts)**.

Що таке Kaniko?

Kaniko — це утиліта, яка дозволяє створювати Docker-образи *всередині контейнера без доступу до Docker-демона*.

Kaniko читає Dockerfile, збирає образ і пушить його **напрямку до реєстру** (наприклад, Amazon ECR).



Це безпечніше та зручніше в Kubernetes-середовищах, бо не вимагає запуску Docker-in-Docker або root-доступу.

Чому це важливо?

У Jenkins CI/CD пайплайнах Kaniko дозволяє:

Конфігурація Jenkins у `values.yaml`

Після налаштування інфраструктури, IAM-ролей, сервіс-акаунтів і встановлення Jenkins у кластер настає черга **описати поведінку самого Jenkins**.

Це робиться в Helm-файлі `values.yaml`, який дозволяє не тільки задати базову конфігурацію, але й інтегрувати Jenkins із зовнішніми сервісами, як-от GitHub та Amazon ECR. За допомогою Jenkins Configuration as Code ([JCasC](#)) ми можемо одразу додати необхідні креденшели, створити seed job і автоматично генерувати pipeline.

У нашому випадку Jenkins буде використовувати **Personal Access Token (PAT)** для доступу до GitHub і **Kaniko** — для безпечного білду Docker-образів прямо в Kubernetes, без встановлення Docker-демона. Такий підхід дозволяє легко автоматизувати весь CI/CD-процес: від клонування репозиторію до доставки образу в **Amazon Elastic Container Registry (ECR)**

```
controller:
  admin:
    username: admin
    password: admin123

  serviceType: LoadBalancer
  servicePort: 80
  service:
    port: 80
    targetPort: 8080

  resources:
    limits:
      cpu: "500m"
      memory: "1Gi"
    requests:
      cpu: "250m"
      memory: "512Mi"

  persistentVolume:
    enabled: true
    storageClass: "ebs-sc"
    size: 10Gi

  installPlugins:
    - kubernetes:latest
    - workflow-aggregator:latest
    - git:latest
    - configuration-as-code:latest
    - credentials-binding:latest
    - github:latest
    - docker-plugin:latest
    - docker-workflow:latest
    - job-dsl:latest

  serviceAccount:
    name: jenkins-sa
    create: false

  JCasC:
    configScripts:
      credentials: |
```

```

credentials:
  system:
    domainCredentials:
      - credentials:
          - usernamePassword:
              scope: GLOBAL
              id: github-token
              username: AndriyDmitriv
              password:
github_pat_11ALJ7SJA0lygqPn0iNuIn_HvIS5uxPUqpOL4uIovR7fmVxTV4M9L5ng
1vTgCdNWFHRJI2HQJC8GzOMFnA
              description: GitHub PAT
seed-job: |
  jobs:
    - script: >
      job('seed-job') {
        description('Job to generate pipeline for Django
project')

        scm {
          git {
            remote {

url('<https://github.com/AndriyDmitriv/infra.git>')
              credentials('github-token')
            }
            branches('*/main')
          }
        }
        steps {
          dsl {
            text(''
              pipelineJob("goit-django-docker") {
                definition {
                  cpsScm {
                    scm {
                      git {
                        remote {
                          url("<https://github.com/AndriyDmitriv/infra.git>")
                            credentials("github-token")
                          }
                        branches("*/main")
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }

```

```

    }
  }
}
'''
}
}
}

```

serviceAccount

```

serviceAccount:
  name: jenkins-sa
  create: false

```

Пояснення:

`name: jenkins-sa` — ім'я сервіс-акаунту, який буде використовуватись Jenkins-контролером для запуску job'ів.

`create: false` — Helm не створює цей акаунт автоматично, бо ми вже створили `jenkins-sa` вручну через Terraform і зв'язали його з IAM-роллю через IRSA. Це потрібно для того, щоб Jenkins міг використовувати Kaniko з доступом до ECR.

JCasC.configScripts.credentials

Це секція Jenkins Configuration as Code (JCasC), яка описує Jenkins credentials.

Безпечніший і простіший спосіб інтеграції з GitHub, особливо для публічних Jenkins-агентів.

```

JCasC:
  configScripts:
    credentials: |
      credentials:
        system:
          domainCredentials:
            - credentials:
                - usernamePassword:

```



```
scope: GLOBAL
id: github-token
username: AndriyDmitriv
password: github_pat_xxx
description: GitHub PAT
```

Пояснення:

Ми додаємо usernamePassword тип credentials, де:

id: github-token — унікальний ID, який використовуватиметься в pipeline/job для доступу.

username — GitHub юзернейм.

password — GitHub PAT (Personal Access Token), який дозволяє Jenkins автентифікуватись до приватного репозиторію без SSH-ключів.

JCasC.configScripts.seed-job

Це автоматична генерація job'ів через **Job DSL plugin**, ****що ****дозволяє автоматично налаштовувати CI/CD при деплоїменті Jenkins.

```
seed-job: |
  jobs:
    - script: >
      job('seed-job') {
        description('Job to generate pipeline for Django
project')

        scm {
          git {
            remote {

url('<https://github.com/AndriyDmitriv/infra.git>')
              credentials('github-token')
            }
            branches('*/main')
          }
        }
        steps {
          dsl {
            text(''
              pipelineJob("goit-django-docker") {
                definition {
```

```

        cpsScm {
            scm {
                git {
                    remote {
                        url("
<https://github.com/AndriyDmitriv/infra.git>")
                        credentials("github-token")
                    }
                    branches("*/main")
                }
            }
        }
    }
}
'''
}
}
}

```

Пояснення:

Seed job:

Клонує `infra` репозиторій.

Використовує `github-token` `credentials` для доступу.

Створює Jenkins pipeline `goit-django-docker`, який читає `Jenkinsfile` із того самого репозиторію.

Конфігурація `Jenkinsfile` у Django-репозиторії

Тепер, коли Jenkins повністю налаштований у кластері, має доступ до GitHub і отримав необхідні IAM-права для пушу в Amazon ECR, залишилось найважливіше — сам `Jenkinsfile`.

Це основа для CI/CD-процесу, яка визначає, що саме Jenkins буде робити після запуску job'и.

Ми використаємо Jenkins pipeline декларативного типу (`pipeline {}`), щоб створити Docker-образ із Django-застосунком і завантажити його до ECR. Для цього застосовуємо контейнер Kaniko — спеціальний інструмент, який дозволяє збирати Docker-образи без `docker daemon` та ідеально підходить для Kubernetes-середовищ.

Контейнер Kaniko запускатиметься всередині окремого Jenkins pod'a, який створюється динамічно. Завдяки `serviceAccountName: jenkins-sa` та попередньо налаштованим IAM-ролям він зможе автентифікуватися в ECR і пушити туди результат збірки.

```
pipeline {
  agent {
    kubernetes {
      yml """
apiVersion: v1
kind: Pod
metadata:
  labels:
    some-label: jenkins-kaniko
spec:
  serviceAccountName: jenkins-sa
  containers:
    - name: kaniko
      image: gcr.io/kaniko-project/executor:v1.16.0-debug
      imagePullPolicy: Always
      command:
        - sleep
      args:
        - 99d
      """
    }
  }

  environment {
    ECR_REGISTRY = "397114334021.dkr.ecr.us-west-2.amazonaws.com"
    IMAGE_NAME   = "app"
    IMAGE_TAG    = "latest"
  }

  stages {
    stage('Build & Push Docker Image') {
      steps {
        container('kaniko') {
          sh '''
            /kaniko/executor \\\
            --context `pwd` \\\
            --dockerfile `pwd`/Dockerfile \\\
            --destination=$ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG \\\
            --cache=true \\\
          '''
        }
      }
    }
  }
}
```


serviceAccountName: jenkins-sa — використовує попередньо створений сервіс-акаунт, який має IAM-роль із правами на пуш до ECR через IRSA (IAM Roles for Service Accounts).

containers:

- **name: kaniko**
- image: gcr.io/kaniko-project/executor:v1.16.0-debug**
- imagePullPolicy: Always**
- command:**
 - **sleep**
- args:**
 - **99d**

image: gcr.io/kaniko-project/executor:v1.16.0-debug — офіційний debug-образ Kaniko. Має всі необхідні інструменти та підтримку логів.

command: ["sleep"], args: ["99d"] — запускає контейнер у пасивному режимі, щоб Jenkins міг увійти в нього та виконати команди через `container('kaniko')`.

```
environment {  
  ECR_REGISTRY = "397114334021.dkr.ecr.us-west-2.amazonaws.com"  
  IMAGE_NAME   = "app"  
  IMAGE_TAG    = "latest"  
}
```

environment — змінні оточення, які використовуються для складання повного шляху до ECR-репозиторію (`$ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG`).

```
stage('Build & Push Docker Image') {  
  steps {  
    container('kaniko') {  
      sh '''  
        /kaniko/executor \  
          --context `pwd` \  
          --dockerfile `pwd`/Dockerfile \  
          --destination=$ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG \  
        '''  
      }  
    }  
  }  
}
```

```

        --cache=true \\
        --insecure \\
        --skip-tls-verify
    ...
}
}
}

```

`stage('Build & Push Docker Image')` — єдина стадія пайплайну, де відбувається білд і пуш.

`container('kaniko')` — Jenkins входить у контейнер `kaniko`, що створений у `pod'i`.

`/kaniko/executor` — головна команда, яка виконує білд образу.

`-context \\pwd``` — поточна директорія Jenkins workspace використовується як контекст збірки (місце, де лежить `Dockerfile`).

`-dockerfile \\pwd/Dockerfile` — шлях до `Dockerfile`.

`-destination=...` — повний шлях до образу в ECR (реєстр + ім'я + тег).

`-cache=true` — дозволяє кешування шарів Docker-образу, пришвидшує повторні збірки.

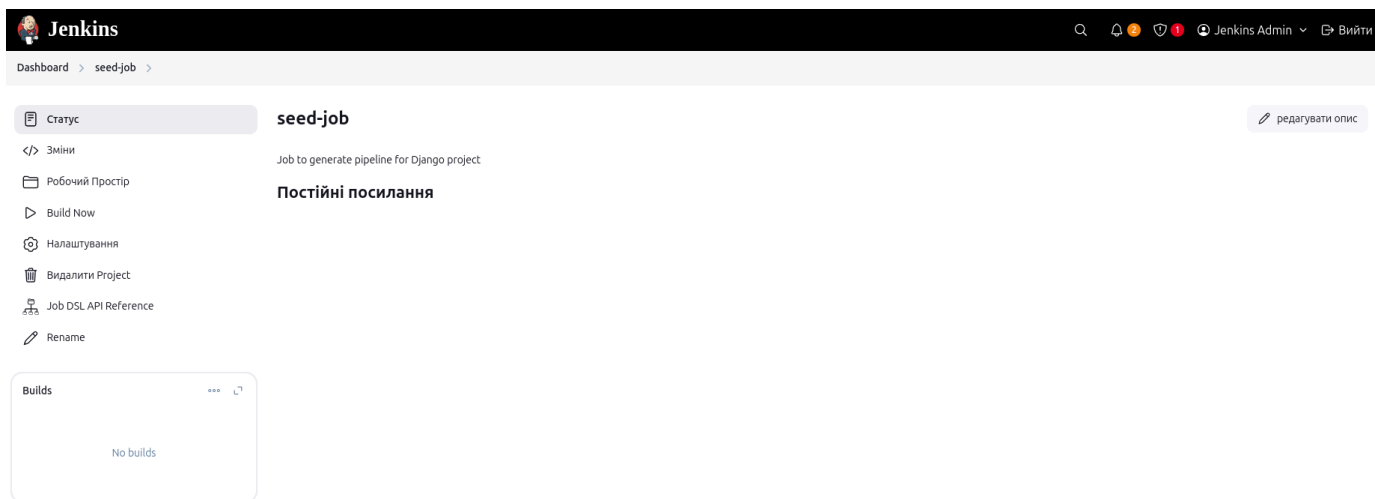
`-insecure --skip-tls-verify` — вимикають TLS-перевірку (може бути корисно при тестуванні, хоча у продакшн — краще прибрати).

Супер. Запускаємо нашу конфігурацію та заходимо в наш Jenkins UI.

The screenshot shows the Jenkins Dashboard interface. On the left, there's a sidebar with navigation links: '+ Новий Item', 'Історія побудов', 'Manage Jenkins', and 'My Views'. Below these are filters for 'Черга побудов' (0/0), 'Статус виконавців побудов' (0/0), and 'Cloud Statistics'. The main area displays a table of jobs. A single job, 'seed-job', is listed with a status of 'N/A' for both 'Востаннє успішно' and 'Востаннє невдало', and 'N/A' for 'Тривалість останньої побудови'. The job is represented by a sun icon. At the bottom of the table, there are checkboxes for 'S', 'M', and 'L'.

S	W	Name	Востаннє успішно	Востаннє невдало	Тривалість останньої побудови
		seed-job	N/A	N/A	N/A

На зображенні бачимо, що з'явився наш пайплайн `seed-job`. Відкриємо його:



Опис: «*Job to generate pipeline for Django project*»

Меню ліворуч:

Статус — загальний огляд

Зміни, Робочий Простір — відображення змін / файлів

Build Now — ручний запуск

Налаштування — конфігурація цієї джоби

Видалити Project — видалення

Job DSL API Reference — доступ до документації DSL

Rename — перейменування

Build History (справа внизу): ще немає запусків (No builds)

Кнопка `редагувати опис` — можна змінити опис джоби

Спробуємо запустити джобу Build Now і глянемо, що відбудеться.

У підсумку ця конфігурація дозволяє повністю автоматизувати CI/CD- процес для збірки та пушу Docker-образу в Amazon ECR без використання Jenkins UI.

Ми створюємо сервіс-акаунт із роллю IAM, що має доступ до ECR, і використовуємо Kaniko в Kubernetes-поді як інструмент білду.

Весь пайплайн описано у `Jenkinsfile` та ініціалізовано через `seed-job`, що також заданий у коді.

Завдяки використанню Helm-чарта, Jenkins і конфігурацій через JCasC ми досягаємо повної декларативності.

